Full Stack Web Development — Lab Sheet 6

Solved: APIs Practical — Full Backend Integration: Node.js + Express.js + REST APIs (No Database)

Source basis: the uploaded Lab Sheet 6, which requires a Node.js/Express REST API, middleware, authentication, testing, a plain JavaScript client, and a React client. ■filecite■turn0file0■L5-L19■

Project structure

```
api-lab/
■■■ server/
■   ■■■ server.js
■   ■■■ package.json
■   ■■■ .env
■   ■■■ .gitignore
■■■ client/
■   ■■■ index.html
■   ■■■ script.js
■■■ client-react/
■   ■■■ src/
■   ■   ■■■ App.jsx
■   ■   ■■■ main.jsx
■   ■■■ .env
■   ■■■ package.json
■■■ API-Lab.postman_collection.json
```

Q1. Server Setup & Building the REST API

The lab sheet asks for the api-lab/server and client folders, Express/CORS/dotenv, PORT=5000, five CRUD routes, a global logger, and an x-api-key middleware for write operations. ■filecite■turn0file0■L10-L19■

Step 1 — Create and install packages

```
mkdir api-lab
cd api-lab
mkdir server client
cd server
npm init -y
npm install express cors dotenv
npm install --save-dev nodemon
```

Step 2 — server/.env

```
PORT=5000
API_KEY=mysecret123
```

Step 3 — server/.gitignore

```
node_modules/
.env
```

Step 4 — server/package.json scripts

```
"scripts": {
  "start": "node server.js",
  "dev": "nodemon server.js"
}
```

Step 5 — server/server.js

```
const express = require("express");
const cors = require("cors");
require("dotenv").config();

const app = express();
const PORT = process.env.PORT || 5000;
const API_KEY = process.env.API_KEY;

app.use(cors());
app.use(express.json());

// Global logger middleware
app.use((req, res, next) => {
```

```
  console.log(`[${new Date().toISOString()}] ${req.method} ${req.originalUrl}`);
  next();
});

// Authentication middleware for write operations
function authCheck(req, res, next) {
  if (req.headers["x-api-key"] !== API_KEY) {
    return res.status(401).json({
      error: "Unauthorized",
      message: "Valid x-api-key header is required"
    });
  }
  next();
}

let tasks = [
  { id: 1, title: "Learn Node.js", completed: false },
  { id: 2, title: "Build Express API", completed: true },
  { id: 3, title: "Test REST endpoints", completed: false },
  { id: 4, title: "Create JavaScript client", completed: false },
  { id: 5, title: "Create React client", completed: false }
];

// GET all
app.get("/api/tasks", (req, res) => {
  res.json(tasks);
});

// GET one
app.get("/api/tasks/:id", (req, res) => {
  const id = Number(req.params.id);
  const task = tasks.find(t => t.id === id);

  if (!task) {
    return res.status(404).json({ error: "Task not found" });
  }
  res.json(task);
});

// POST
app.post("/api/tasks", authCheck, (req, res) => {
  const { title, completed = false } = req.body;

  if (!title || typeof title !== "string") {
    return res.status(400).json({ error: "title is required" });
  }

  const newTask = {
    id: tasks.length ? Math.max(...tasks.map(t => t.id)) + 1 : 1,
    title,
    completed: Boolean(completed)
  };

  tasks.push(newTask);
  res.status(201).json(newTask);
});

// PUT
app.put("/api/tasks/:id", authCheck, (req, res) => {
  const id = Number(req.params.id);
  const index = tasks.findIndex(t => t.id === id);

  if (index === -1) {
    return res.status(404).json({ error: "Task not found" });
  }

  const { title, completed } = req.body;
  tasks[index] = {
    ...tasks[index],
    ...(title !== undefined ? { title } : {}),
    ...(completed !== undefined ? { completed: Boolean(completed) } : {})
  };

  res.json(tasks[index]);
});

// DELETE
app.delete("/api/tasks/:id", authCheck, (req, res) => {
  const id = Number(req.params.id);
  const index = tasks.findIndex(t => t.id === id);

  if (index === -1) {
    return res.status(404).json({ error: "Task not found" });
  }
```

```
  const deleted = tasks.splice(index, 1)[0];
  res.json({ message: "Task deleted", task: deleted });
});

// 404 handler
app.use((req, res) => {
  res.status(404).json({
    error: "Not Found",
    message: `Route ${req.method} ${req.originalUrl} does not exist`
  });
});

// Centralized error handler
app.use((err, req, res, next) => {
  console.error(err);
  res.status(err.status || 500).json({
    error: "Server Error",
    message: err.message || "Something went wrong"
  });
});

app.listen(PORT, () => {
  console.log(`API server running at http://localhost:${PORT}`);
});
```

Q2. Securing, Testing & Error-Handling the API

CORS should be enabled during development. The collection should test all five endpoints, including x-api-key for POST/PUT/DELETE, and the server needs both unmatched-route and centralized error handlers. ■filecite■turn0file0■L20-L27■

CORS test

Temporarily comment out app.use(cors()), call the API from the browser client, and observe a browser CORS-blocked request. Re-enable app.use(cors()) and reload the client. The exact browser wording varies by browser, so record the message shown in your own console.

Run server

```
cd server
npm run dev
```

Postman / Thunder Client test data

```
GET    http://localhost:5000/api/tasks
GET    http://localhost:5000/api/tasks/1

POST   http://localhost:5000/api/tasks
Header: x-api-key: mysecret123
Header: Content-Type: application/json
Body:
{
  "title": "New API Task",
  "completed": false
}

PUT    http://localhost:5000/api/tasks/1
Header: x-api-key: mysecret123
Header: Content-Type: application/json
Body:
{
  "title": "Updated Task",
  "completed": true
}

DELETE http://localhost:5000/api/tasks/1
Header: x-api-key: mysecret123
```

Expected security result: GET works without the key; POST/PUT/DELETE return HTTP 401 when x-api-key is missing or incorrect. ■filecite■turn0file0■L23-L25■

Sample successful POST response

```
HTTP 201
{
  "id": 6,
  "title": "New API Task",
  "completed": false
}
```

Q3. Consuming the API — Plain JavaScript Client

The plain JavaScript client must fetch GET /api/tasks, render task titles, show Loading..., and display a readable error if the API fails. ■filecite■turn0file0■L28-L34■

client/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Task Manager</title>
</head>
<body>
  <h1>Task Manager</h1>
  <p id="status">Loading...</p>
  <ul id="taskList"></ul>
  <script src="script.js"></script>
</body>
</html>
```

client/script.js

```
const status = document.getElementById("status");
const taskList = document.getElementById("taskList");

async function loadTasks() {
  status.textContent = "Loading...";
  taskList.innerHTML = "";

  try {
    const response = await fetch("http://localhost:5000/api/tasks");

    if (!response.ok) {
      throw new Error(`HTTP ${response.status}`);
    }

    const tasks = await response.json();

    status.textContent = "";
    tasks.forEach(task => {
      const li = document.createElement("li");
      li.textContent =
        `${task.title} — ${task.completed ? "Completed" : "Pending"}`;
      taskList.appendChild(li);
    });
  } catch (error) {
    status.textContent =
      `Unable to load tasks: ${error.message}`;
  }
}

loadTasks();
```

Failure-path test: stop the Node server while the page is open. The catch block should show an error message instead of leaving a blank/broken page.

Q4. Consuming the API — React Client

The React client must implement loading/error/data states, demonstrate an axios version, POST a new task, and delete tasks without a page reload. ■filecite■turn0file0■L35-L43■

Create the React app

```
cd api-lab
npm create vite@latest client-react -- --template react
cd client-react
npm install
npm install axios
npm run dev
```

client-react/.env

```
VITE_API_URL=http://localhost:5000/api
```

client-react/src/App.jsx

```
import { useEffect, useState } from "react";
import axios from "axios";

const API_URL = import.meta.env.VITE_API_URL;
```

```
export default function App() {
  const [tasks, setTasks] = useState([]);
  const [title, setTitle] = useState("");
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");

  async function loadTasks() {
    setLoading(true);
    setError("");

    try {
      const response = await fetch(`${API_URL}/tasks`);
      if (!response.ok) {
        throw new Error(`HTTP ${response.status}`);
      }
      const data = await response.json();
      setTasks(data);
    } catch (err) {
      setError(`Unable to load tasks: ${err.message}`);
    } finally {
      setLoading(false);
    }
  }

  useEffect(() => {
    loadTasks();
  }, []);

  // Axios version:
  // const response = await axios.get(`${API_URL}/tasks`);
  // const data = response.data;
  // Axios parses JSON automatically and rejects non-2xx responses.

  async function addTask(e) {
    e.preventDefault();
    if (!title.trim()) return;

    try {
      const response = await fetch(`${API_URL}/tasks`, {
        method: "POST",
        headers: {
          "Content-Type": "application/json",
          "x-api-key": "mysecret123"
        },
        body: JSON.stringify({
          title: title.trim(),
          completed: false
        })
      });

      const data = await response.json();

      if (!response.ok) {
        throw new Error(data.message || data.error || `HTTP ${response.status}`);
      }

      setTasks(prev => [...prev, data]);
      setTitle("");
      setError("");
    } catch (err) {
      setError(`Could not create task: ${err.message}`);
    }
  }

  async function deleteTask(id) {
    try {
      const response = await fetch(`${API_URL}/tasks/${id}`, {
        method: "DELETE",
        headers: {
          "x-api-key": "mysecret123"
        }
      });

      const data = await response.json();

      if (!response.ok) {
        throw new Error(data.message || data.error || `HTTP ${response.status}`);
      }

      setTasks(prev => prev.filter(task => task.id !== id));
      setError("");
    } catch (err) {
      setError(`Could not delete task: ${err.message}`);
    }
```

```
  }

  if (loading) return <p>Loading...</p>;

  return (
    <div style={{ maxWidth: 650, margin: "40px auto", fontFamily: "Arial" }}>
      <h1>Task Manager Mini App</h1>

      {error && (
        <p style={{ padding: 10, border: "1px solid #ccc" }}>
          {error}
        </p>
      )}

      <form onSubmit={addTask}>
        <input
          value={title}
          onChange={e => setTitle(e.target.value)}
          placeholder="Enter task title"
        />
        <button type="submit">Add Task</button>
      </form>

      <ul>
        {tasks.map(task => (
          <li key={task.id} style={{ margin: "10px 0" }}>
            {task.title} — {task.completed ? "Completed" : "Pending"}
            <button
              onClick={() => deleteTask(task.id)}
              style={{ marginLeft: 10 }}
            >
              Delete
            </button>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

client-react/src/main.jsx

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

Q5. Full-Stack Error Handling, Config & Final Integration

The final task is to show readable 404/401 errors, move configuration into .env files, use VITE_API_URL in React, and combine the backend and frontend into the Task Manager Mini App. ■filecite■turn0file0■L44-L53■

Important correction for the React client

For a coursework submission, do not hardcode the API key in production code. The lab specifically asks for configuration values to be moved into configuration. For a local practical, the API key can be supplied through a development mechanism; the server must remain the authority that checks it.

Recommended final React configuration

```
client-react/.env
VITE_API_URL=http://localhost:5000/api
```

For the simple lab implementation above, the API key is shown directly because the lab asks for the x-api-key header. For a real application, never expose a secret API key in a browser environment variable, because VITE_* values are client-visible.

Final testing checklist

```
1. Start server:       cd server && npm run dev
2. Check GET tasks:     GET http://localhost:5000/api/tasks
3. Check GET one:       GET http://localhost:5000/api/tasks/1
4. POST without key:    expect 401
5. POST with key:       expect 201
6. PUT with key:        expect 200
7. DELETE with key:     expect 200
```